

Setting Up with *Spring Initializr*

Time to build! We'll start by using Spring Initializr to generate our first Spring Boot project. This guide walks through every field on the page so you understand exactly what each choice means and why we're picking it.

- ▶ Project setup guide ▶ Tool: <https://start.spring.io>
- ▶ Outcome: A ready-to-run Spring Boot project

01 What We're Doing

In this lesson, we'll use **Spring Initializr** to generate a Spring Boot project. Think of Spring Initializr as a **project generator** — you tell it what you want, and it creates a ready-to-open project with the correct folder structure, build file, and dependencies.



Think of it like ordering a pizza online

You go to the website, choose your **size** (project type), **crust** (language), **toppings** (dependencies), and click **order** (generate). A few seconds later, your custom pizza (project) is ready for pickup (download). You don't bake it from scratch — the kitchen does the work.

Big Picture: This is where everything we've learned comes together. We're creating a **microservice** (lesson 1) that serves a **REST API** (lesson 2), built with the **Spring Framework** (lesson 3) using tools from the **Spring Ecosystem** (lesson 4), powered by **Spring Boot** (lesson 5). Now we build it.

What We'll Build

A **Product Service** microservice — a REST API that manages products. We'll configure the project in this session, then write the actual code in the next one.

02 Open Spring Initializr

✓ DO THIS NOW

Open your browser and go to:

<https://start.spring.io>

You'll see a form with several fields. Let's go through **every single one** so you understand what you're choosing and why.

SPRING INITIALIZR — ACTUAL PAGE LAYOUT

spring

initializr

Project

☐ Gradle – Groovy

☐ Gradle – Kotlin

☒ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT)

☐ 4.1.0 (RC1)

☐ 4.0.7 (SNAPSHOT)

☒ 4.0.6

☐ 3.5.15 (SNAPSHOT)

☐ 3.5.14

Project Metadata

Group

Artifact

Package name

Packaging

☒ Jar

☐ War

Configuration

☒ Properties

☐ YAML

Dependencies

ADD DEPENDENCIES...

CTRL + B

No dependency selected

GENERATE

CTRL + G

EXPLORE

CTRL + SPACE

...

Now let's explain each field one by one.

03 Project Type – Maven or Gradle

Project

This is your **build tool** – the tool that compiles your code, manages dependencies, runs tests, and packages your application into a JAR file. You'll see three options.

Our choice: Maven

OPTION	WHAT IT IS	BUILD FILE
Gradle – Groovy	Gradle build tool using Groovy syntax for the build script	<code>build.gradle</code>

OPTION	WHAT IT IS	BUILD FILE
Gradle – Kotlin	Gradle build tool using Kotlin syntax for the build script	<code>build.gradle.kts</code>
Maven	The most widely used build tool in enterprise Java	<code>pom.xml</code> (XML)

ASPECT	MAVEN	GRADLE
Style	Declarative — you describe what you need	Programmatic — you can write logic
Learning curve	Easier for beginners	More flexible but more complex
Industry usage	Most common in enterprise Java	Growing, popular in Android and newer projects
Speed	Slower on large projects	Faster with incremental builds

Why Maven? It's the most widely used build tool in Java enterprise projects. Most tutorials, documentation, and job postings assume Maven. Once you understand Maven, learning Gradle later is straightforward.

✅ DO THIS NOW

Select **Maven** (it should be selected by default).

04 Language — Java, Kotlin, or Groovy

Language

The programming language you'll write your code in. All three run on the JVM (Java Virtual Machine) and can use Spring Boot.

Our choice: Java

LANGUAGE	DESCRIPTION	WHEN TO USE
Java	The original JVM language. Most widely used, most resources, most job opportunities.	Default choice. What we're learning.
Kotlin	Modern language by JetBrains. More concise than Java. Fully interoperable with Java.	Popular for Android and modern Spring projects.
Groovy	Dynamic language on the JVM. Used mostly for scripting and Gradle build files.	Rarely used for Spring Boot applications.

✅ DO THIS NOW

Select **Java** (it should be selected by default).

05 Spring Boot Version

Spring Boot

Which version of Spring Boot to use. You'll see several options — some marked as SNAPSHOT or M1/RC.

Our choice: Latest stable version (no SNAPSHOT, no M/RC label)

What Do the Labels Mean?

LABEL	MEANING	SHOULD YOU USE IT?
SNAPSHOT	In active development. Changes daily. May have bugs.	✗ Never for real projects
M1, M2 (Milestone)	Pre-release for testing new features. Not final.	✗ Only for trying new features
RC1 (Release Candidate)	Almost final. Feature-complete but may have minor issues.	⚠ Only if you need a specific new feature
No label (e.g., 3.3.6)	Stable release. Tested, production-ready.	✓ Always use this

Rule: Always pick the latest version that has **no label** next to it. That's the stable, production-ready release. On the page right now you'll see versions like 4.1.0 (SNAPSHOT), 4.1.0 (RC1), 4.0.7 (SNAPSHOT), **4.0.6**, 3.5.15 (SNAPSHOT), and 3.5.14. Pick **4.0.6** — it's the latest stable.

Note: The versions on the page change over time as Spring releases updates. When you open the page, the exact numbers might be different. Just apply the same rule: pick the **highest version without a label**.

✓ DO THIS NOW

Select the **latest stable version** — the one without SNAPSHOT, M, or RC next to it.

06 Project Metadata

This section defines **who you are** and **what your project is called**. These values end up in your `pom.xml` and determine your package structure. You'll see three visible fields by default.

Group

Your organization's domain name in **reverse**. This is a Java convention (from the Java package naming standard) to ensure unique package names worldwide.

Our choice: `com.example`

Group ID Naming Convention

The Group ID follows the **reverse domain name** convention. Take your organization's website domain and reverse it:

YOUR DOMAIN	GROUP ID	WHY
example.com	<code>com.example</code>	Standard placeholder for training/demos
google.com	<code>com.google</code>	Google's real Group ID
netflix.com	<code>com.netflix</code>	Netflix's real Group ID
techcorp.ph	<code>ph.techcorp</code>	Philippine domain — country code comes first
mycompany.co.uk	<code>uk.co.mycompany</code>	UK domain — reverse all parts
university.edu	<code>edu.university</code>	Education domain
openproject.org	<code>org.openproject</code>	Organization domain

Rules for Group ID

- Always **lowercase** — `com.example` not `Com.Example`
- Use **dots** to separate parts — `com.example` not `com-example`
- Must be a **valid Java package name** — no spaces, no special characters, no hyphens
- Cannot start with a **number** — `com.123company` is invalid

- Avoid Java **reserved words** — don't use `com.new` or `com.class`
- Can add **sub-groups** for large organizations — `com.google.cloud` , `com.google.android`

Common mistakes: Don't use `www` in the Group ID — `com.example` not `com.www.example`.
Don't use your personal name unless it's a personal project. Don't invent a domain you don't own for production projects (but `com.example` is fine for training).

Artifact

The **name of your project**. This becomes the name of your JAR file (e.g, `product-service-0.0.1-SNAPSHOT.jar`) and the folder name. Use lowercase with hyphens.

Our choice: `product-service`

Package name

The **base Java package** for your code. Auto-generated from Group + Artifact. This becomes the folder structure: `com.example.productservice` → `com/example/productservice/`.

Our choice: `com.example.productservice` (auto-generated)

How these connect: Package name = Group + Artifact concatenated. Group (`com.example`) + Artifact (`product-service`) = Package name (`com.example.productservice`). Notice the hyphen is **removed** — Java package names cannot contain hyphens.

⚠ **Watch out for hyphens!** The Artifact can have hyphens (`product-service`) because it's just a project name. But the Package name **cannot** — Java package names only allow letters, numbers, and dots. When you type a hyphenated Artifact, the Initializr *should* auto-remove the hyphen in the Package name. But **always double-check!**

ARTIFACT	✗ WRONG PACKAGE NAME	✓ CORRECT PACKAGE NAME
product-service	com.example.product-service	com.example.productservice
order-management	com.example.order-management	com.example.ordermanagement
user-api	com.example.user-api	com.example.userapi

If the Package name field still has a hyphen, **manually remove it** before generating. A hyphen in the package name will cause a compilation error.

✓ DO THIS NOW

Fill in the metadata:

→ **Group:** `com.example`

→ **Artifact:** `product-service`

→ **Package name:** verify it says `com.example.productservice` — **no hyphens!** If it shows `com.example.product-service`, manually remove the hyphen.

07 Packaging — JAR or WAR

Packaging

How your application will be packaged for deployment.

Our choice: Jar

PACKAGING	WHAT IT IS	WHEN TO USE
JAR (Java Archive)	A stand-alone executable file. Includes the embedded Tomcat server. Run with <code>java -jar app.jar</code> .	✅ Always for microservices and modern Spring Boot apps
WAR (Web Application Archive)	Must be deployed to an external server (like a standalone Tomcat installation). The old way.	Only when you must deploy to an existing external server (legacy requirement)

🔗 **Microservices Connection:** Remember Spring Boot feature #2 — "Embed Tomcat directly, no need to deploy WAR files"? That's why we choose JAR. Each microservice is a self-contained JAR with its own embedded server. No external Tomcat to install or manage.

✅ DO THIS NOW

Select **Jar** (it should be selected by default).

08 Configuration — Properties or YAML

Configuration

The format for your configuration file. This determines whether your project generates `application.properties` or `application.yml`.

Our choice: Properties

FORMAT	FILE GENERATED	SYNTAX STYLE	EXAMPLE
Properties	<code>application.properties</code>	Simple key=value pairs, one per line	<code>server.port=8081</code>

FORMAT	FILE GENERATED	SYNTAX STYLE	EXAMPLE
YAML	application.yml	Indentation-based hierarchy	server: port: 8081

Both formats do the exact same thing — they just look different. Here's a side-by-side comparison:

PROPERTIES FORMAT

```
server.port=8081
spring.application.name=product-service
spring.datasource.url=jdbc:mysql://localhost:3306/products_db
spring.datasource.username=root
```

YAML FORMAT (SAME CONFIG)

```
server:
  port: 8081
spring:
  application:
    name: product-service
  datasource:
    url: jdbc:mysql://localhost:3306/products_db
    username: root
```

Why Properties? It's simpler for beginners — one line per setting, no indentation rules to worry about. YAML looks cleaner for deeply nested configuration but is sensitive to spacing (a wrong indent breaks everything). Most Spring Boot tutorials use Properties. You can always switch later.

✅ DO THIS NOW

Select **Properties** (it should be selected by default).

About Java version: In earlier versions of the Initializr, there was a Java version selector on the main page (17, 21, 23). This may be hidden or set automatically based on your Spring Boot version. If you see it, select **17** or **21** to match what's installed on your machine (check with `java -version` in your terminal). If you don't see it, Spring Boot picks a sensible default — Java 17 is the minimum for Spring Boot 3.x and 4.x.

09 Dependencies — The Most Important Part

This is where you choose **what your application can do**. Each dependency adds a Spring Boot starter (a bundle of libraries) to your project. Click "ADD DEPENDENCIES" or use the search box.

Remember from the Spring Boot lesson: Starters are pre-packaged bundles of compatible libraries. Adding "Spring Web" gives you Spring MVC + embedded Tomcat + Jackson (JSON) — all in one click.

Dependencies We'll Add

1. Spring Web

Adds `spring-boot-starter-web`. This gives us everything we need to build REST APIs: Spring MVC for handling HTTP requests, embedded Tomcat as the web server, and Jackson for automatic JSON conversion.

Starter: `spring-boot-starter-web`

 **Without this:** Your application has no web server and can't handle HTTP requests. No REST API. This is the most essential dependency for any microservice.

2. Spring Data JPA

Adds `spring-boot-starter-data-jpa`. This gives us Spring Data JPA + Hibernate for database access. Define an interface, get CRUD operations automatically — no SQL needed for basic operations.

Starter: `spring-boot-starter-data-jpa`

 **Without this:** You'd have to write raw JDBC code with manual SQL queries, connection management, and result set parsing. Spring Data JPA eliminates all of that boilerplate.

3. MySQL Driver

Adds the MySQL JDBC driver (`mysql-connector-j`). This is the actual driver that lets Java talk to a MySQL database. Spring Data JPA is the framework; this is the **bridge to the specific database**.

Dependency: `com.mysql:mysql-connector-j`

Using a different database? Swap this for the appropriate driver: PostgreSQL Driver for PostgreSQL, H2 Database for an in-memory test database, or Oracle Driver for Oracle. The rest of your code stays the same — only the driver and connection properties change.

4. Spring Boot DevTools

Adds `spring-boot-devtools`. Enables **automatic restart** when you change code during development. Save a file → app restarts in 1–2 seconds. Also enables live reload for browser-based testing.

Dependency: `spring-boot-devtools` (scope: runtime only – not in production)

DevTools is development-only. It's automatically disabled when you run the app as a packaged JAR in production. It only works when you run from your IDE or with `mvn spring-boot:run`. Safe to include — it won't affect production.

What We're NOT Adding (and Why)

DEPENDENCY	WHY NOT NOW
Lombok	We're writing getters/setters manually to understand the fundamentals first
Spring Security	Adds authentication to every endpoint immediately — we'd get locked out. Add it later when we're ready for security.
Spring Boot Actuator	Useful but not needed for our first project. We'll add it later.
Validation	We'll add input validation after we get the basic CRUD working.

⚠ **Common mistake:** Don't add Spring Security to a new project unless you're ready to configure it. By default, it protects ALL endpoints with a generated password. You'll see a login page instead of your API and get confused. Add it only when you're intentionally implementing authentication.

✅ **DO THIS NOW**

Click **"ADD DEPENDENCIES"** (or press Ctrl+B) and search for each one:

1 Search **"Spring Web"** → click to add

2 Search **"Spring Data JPA"** → click to add

3 Search **"MySQL Driver"** → click to add

4 Search **"Spring Boot DevTools"** → click to add

You should see 4 dependencies listed on the right side of the page.

10 Verify Your Settings — Quick Reference

That's everything you need to know about the Spring Initializr form. Before we move on, here's a summary of the settings we discussed. You'll use this as your reference when we generate the project in the laboratory exercise.

FIELD	YOUR VALUE
Project	Maven
Language	Java
Spring Boot	Latest stable (e.g., 4.0.6 — no SNAPSHOT/M/RC)
Group	<code>com.example</code>
Artifact	<code>product-service</code>
Package name	<code>com.example.productservice</code> (no hyphens!)
Packaging	Jar
Java	17 or 21 (must match your installed JDK — check with <code>java -version</code>)
Configuration	Properties
Dependencies	Spring Web, Spring Data JPA, MySQL Driver, Spring Boot DevTools



What we covered today

1. What Spring Initializr is and why we use it
2. Every field on the form: Project type, Language, Boot version, Metadata, Packaging, Configuration
3. Group ID naming conventions (reverse domain, lowercase, no hyphens)
4. Package name rules (no hyphens — always verify!)
5. Why we chose each dependency: Spring Web, Spring Data JPA, MySQL Driver,



Next — Laboratory Exercise: Your First Spring Boot App

In the next session, we'll put this into practice. You will:

1. Generate and download the project from Spring Initializr using the settings above
2. Open the project in your IDE and explore the generated files
3. Configure `application.properties` (database connection, server port)
4. Create the **Product** entity (model)
5. Create the **ProductRepository** (database access)
6. Create the **ProductService** (business logic)
7. Create the **ProductController** (REST endpoints)
8. Run the application and test with Postman or browser
9. See our REST API return real JSON data from the database

Don't worry — we'll walk through everything step by step together.

The learning path: Microservices → REST API → Spring Framework → Spring Ecosystem → Spring Boot → **Spring Initializr (today)** → Lab: Your First Spring Boot App (next)